

Thursday Test Deployment Setup

Speech-to-Speech Pipeline

Progress so far

1. Introduction

This project contains the code for a social robotics experiment where the Misty-II robot guides participants through a series of collaborative problem-solving tasks. One version of the robot behaves ‘pro-actively’, responding to human state cues (e.g., frustration, confusion) and another that follows a rule-based script.

2. Technical Bits

- The AI runs on a desktop PC with an RTX 5070 Ti, a laptop is connected to the PC via tailscale and handles traffic between the remote PC and Misty.

 Hugging Face Speech-to-Speech Pipeline:

VAD → STT → LLM → TTS

1. **Voice Activity Detection:** Silero VAD v5 (real-time speech detection)
2. **Speech-to-Text:** Faster-Whisper (GPU-accelerated transcription)
3. **Language Model:** Microsoft Phi-3-mini-4k (conversational AI with custom role/context/rules)
4. **Text-to-Speech:** Parler-TTS (natural voice synthesis)

Experimental Design

Problem-Solving Tasks

The robot guides participants through **3 collaborative tasks** that revolve around solving a cryptic ‘who-dunnit’ mystery (where is the missing robot?). Tasks include:

1. **‘Suspect’ Identification:** Yes/no questions to identify who took the robot from the lab; 6×4 grid (is perp wearing glasses?, wearing hat? etc.)
2. **System Analysis:** Analyzing robot logs to determine functionality status (misty will help decipher cryptic log codes to determine if robot is online, offline, low battery etc.)
3. **Location Inference:** Using similar logs (e.g., WiFi logs) to find location of the missing robot

Robot Personality

- **Adaptive behavior modes:** CONTROL (rules-based, help-when-asked) vs RESPONSIVE (proactive, responsive)
- **Affect detection:** Monitors user hesitation, confusion, frustration and affect cues embedded in audio and text; will be wiring in more advanced affect detection once pipeline is stable.
- **Progressive hint system:** 3-tier scaffolding (H1→H2→H3)
- **Expression system:** 16 distinct robot expressions linked to conversation context (adding more once pipeline is stable)

JSON Response Architecture

```
{
  "msg": "I'm sensing hesitation—would you like a hint?",
  "expression": "hint" # triggers robot to display 'thinking' expression (RGB
  LED + head movement + LCD Screen + arm movement)
}
```

- **msg**: Speech content for TTS
- **expression**: Robot action trigger (LED, head movement, facial display)

Quick Start (30 seconds)

Step 1: Start Pipeline Server on Desktop PC

```
cd /home/sheron/Documents/robots/speech-to-speech
./start_pipeline_server.sh
# Simple launcher script - checks environment, GPU, and starts server
# Wait for: "Receiver waiting to be connected..."
```

Alternative (manual command):

```
source .venv-312/bin/activate
python s2s_pipeline.py --mode socket --recv_host 0.0.0.0 --send_host 0.0.0.0
--device cuda --stt faster-whisper --llm transformers --tts parler --
lm_model_name microsoft/Phi-3-mini-4k-instruct --log_level info
```

Step 2: Test Connection from Laptop

```
python test_remote_connection.py 100.95.246.30
# Should show: "🚀 All connections successful! 📶 Average network latency:
~4.9ms"
# Desktop PC Tailscale IP: 100.95.246.30
```

Step 3: Connect Misty Robot

```
python misty_integration.py 192.168.0.137 100.95.246.30
# Misty IP: 192.168.0.137 | Desktop PC Tailscale IP: 100.95.246.30
```

Pre-Test Checklist

Desktop PC Setup (All Steps Complete)

- ☐ Python 3.12 environment activated (.venv-312)
- ☐ CUDA drivers working (nvidia-smi shows RTX 5070 Ti)
- ☐ All models cached and loaded (first run may take 2-3 minutes)
- ☐ Firewall allows ports 12345, 12346 (sudo ufw allow 12345 && sudo ufw allow 12346)

- ☐ Pipeline server shows “Server listening on 0.0.0.0:12345”

Laptop Setup (All Steps Complete)

- ☐ `git clone https://github.com/shaunaheron2/misty-project.git`
- ☐ `Python 3.12 pyenv install 3.12` or package manager
- ☐ **Dependencies:** Only need minimal packages for network testing and Misty control:

```
pip install requests socket mistyPy
# Or just use built-in socket module for connection testing
```

- ☐ **Misty SDK:** `pip install git+https://github.com/MistyCommunity/Python-SDK.git`

Network Configuration

- ☐ Desktop PC and laptop connected via Tailscale VPN
- ☐ Desktop PC IP address known (`ip addr show | grep inet`)
- ☐ Misty robot connected to same WiFi as laptop
- ☐ Misty robot IP address known (check Misty web interface)

Robot Preparation

- ☐ Misty robot powered on and responsive
- ☐ Misty SDK installed in Python environment

Technical Highlights

Architecture

- **Distributed Processing:** Separates AI computation from robot interaction
- **Real-time Performance:** Component-level latency monitoring with automatic grading
- **Structured Conversation:** Task-specific prompts with pre-defined knowledge base

Performance Metrics So Far (when not connected to Misty)

```
Component Breakdown:
├─ VAD: ~45ms (voice detection)
├─ STT: ~294ms (speech transcription)
├─ LLM: ~760ms (response generation)
├─ TTS: ~452ms (voice synthesis)
└─ Robot: ~?ms (physical expressions)

Total: < 1701ms (Grade A+ performance)
```

Troubleshooting

Common Issues

1. **“Connection refused”:** Check firewall settings on desktop PC
2. **Models downloading:** First run in new environment rebuilds cache

3. **Robot not responding:** Verify Misty IP and SDK installation
4. **High latency:** Check WiFi network, use ethernet if possible

CRITICAL: Potential Misty Audio Issues

Problem: Misty SDK commands work, but audio input fails (university networks? or onboard STT faulty?)

Symptoms:

- Robot expressions and movements work
- Speech detection from Misty microphone fails (recording speech via misty to .wav works)
- University firewall blocks audio streaming ports?
- problem with Misty's onboard STT?

Solutions (Test Before):

1. **Audio Bypass:** Use laptop microphone instead of Misty's

```
# Modified integration: laptop audio → pipeline → misty expressions
python laptop_audio_misty_Test.py 192.168.0.137 100.95.246.30
# Misty: 192.168.0.137 | Desktop PC: 100.95.246.30 (Tailscale)
```

2. **Text Input Test:** Keyboard input for Test safety

```
python text_input_Test.py 192.168.0.137 100.95.246.30
# Same IPs, but keyboard input instead of speech
```

3. **Audio Port Testing:**

```
# Test Misty's audio capabilities first
python -c "
from mistyPy.Robot import Robot
robot = Robot('192.168.0.137')
robot.start_recording_audio('test.wav')
# Check if recording works on network
"
```

Key IP Addresses:

- **Misty Robot:** 192.168.0.137
- **Desktop PC (Tailscale):** 100.95.246.30
- **Laptop-to-PC Latency:** ~4.9ms (home) | expect 10-50ms (Test site)

Test Strategy: Always have text-input backup ready! I also have a separate laptop setup ready (laptop_demo.config.py) with smaller models if needed. (1050ti on laptop is ready w/cuda installed)

Fallback Options

1. **Local Mode:** Run everything on laptop if network issues (see laptop_demo)
2. **API Mode:** Use cloud services (requires API keys)

Emergency Commands

```
# Check if pipeline is running
ps aux | grep s2s_pipeline

# Kill stuck processes
pkill -f s2s_pipeline

# Quick restart
./laptop_Test.sh
```

VSCode Setup

- **OS:** Debian-based Linux (PikaOS 13.0) on pc and laptop
- **Remote Development:** SSH into desktop PC to edit code while pipeline runs
- **Integrated Terminal:** Run commands and monitor logs in same window
- **Python Support:** Excellent debugging and intellisense for robotics code
- **Git Integration:** Easy commit/push workflow for research iterations

Essential Extensions:

```
# Install VSCode extensions
code --install-extension ms-python.python
code --install-extension ms-vscode-remote.remote-ssh
code --install-extension ms-vscode.remote-explorer
```

Project Workspace Setup:

1. **Open Project:** code /home/sheron/Documents/robots/speech-to-speech
2. **Python Interpreter:** Select Python 3.12 (.venv-312/bin/python)
3. **Terminal Setup:** Multiple terminals for pipeline, monitoring, and git

```
python start_pipeline_server.sh # Terminal 1
```

4. **Remote SSH:** Connect to desktop PC from laptop

```
python test_remote_connection.py
```

Performance Dashboard

Real-time metrics available during Test:

- Component latency breakdown

- End-to-end conversation timing
- Performance grade (A+ to D)
- Network latency monitoring
- Robot expression execution times

Data Export: All metrics saved to JSON for post-Test analysis

Test Success Criteria

- ☐ Total response time < 2000ms consistently
- ☐ All robot expressions working correctly
- ☐ JSON parsing functioning (speech + actions)
- ☐ LLM/Misty sticking to mystery script
- ☐ No crashes or unhandled exceptions
- ☐ Clear audio input/output quality
- ☐ Network latency < 50ms
- ☐ All components logging correctly

This document was compiled with help from Claude Code Sonnet 4.